

2025-2 종합설계기획

EisenValor 졸업작품 기획서

목차

1. 연구목적
 - 1.1. 배경 & 필요성
 - 1.2. 클라이언트
 - 1.3. 서버
 - 1.4. 결론
2. 게임 개요
 - 2.1. 소개
 - 2.1.1. 소개
 - 2.1.2. 재미 요소
 - 2.1.3. 플랫폼 & 권장사양
 - 2.2. 규모
 - 2.2.1. 주요 오브젝트
 - 2.2.2. 상호작용
 - 2.3. 플레이
 - 2.3.1. 카메라
 - 2.3.2. 컨트롤
3. 게임 특징
 - 3.1. 태세 전환
 - 3.2. 스탠스 및 심리전
 - 3.3. 딜레이를 통한 격투 게임스러운 타이밍
 - 3.4. 목표 기반 팀플레이
 - 3.5. NPC 전장 스케일 확보
 - 3.6. 전장 기믹 오브젝트로 전술 변수
 - 3.7. 현실적인 그래픽 연출

4. 게임 디자인
 - 4.1. 맵 구조
 - 4.2. 시스템
 - 4.2.1. 전투태세 및 스탠스
 - 4.2.2. 딜레이 & 스테미나
 - 4.2.3. 봇 장수
 - 4.2.4. NPC 병사
5. 개발 환경 및 활용 방법
 - 5.1. 엔진 및 그래픽
 - 5.2. 개발 도구
 - 5.3. 외부 라이브러리
 - 5.4. 리소스
 - 5.5. 협업 및 관리
6. 타 게임과의 비교(경쟁력)
7. 개인별 준비 현황
8. 중점연구분야
9. 기술적 강점
 - 9.1. 클라이언트
 - 9.2. 서버
10. 개발 일정 및 구성원 역할분담
11. 참고자료

1. 연구목적

1.1. 배경 & 필요성

실시간 멀티플레이 환경에서 사실적인 그래픽과 낮은 지연의 상호작용을 동시에 달성하기 위한 기술 설계 및 구현을 목표로 한다.

이를 위해 클라이언트는 **DirectX12** 및 **DXR** 기반의 최신 렌더링 파이프라인 구축을, 서버는 **Registered I/O** 기반의 고성능 서버 구조를 필요로 한다.

1.2. 클라이언트

사실적인 그래픽을 위해서 **DirectX12** 환경과 그 내의 **DirectX Raytracing(DXR)**를 활용한 **Clustered Forward+**, 실시간 하이브리드 레이트레이싱 렌더링을 잘 결합하여 설계하고 구현할 예정이다.

또한 **Clustered Forward+**의 장점을 살려 **Volumetric Rendering**을 통해 사실적인 안개와 불, 수중 효과를 구현해 낼 것이다.

다음으로, 게임에 많은 애니메이션이 필요하므로 중요도에 따라 프레임 개수를 조정하여 애니메이션 **LOD**를 구현할 것이다. 또 **IK**를 이용해 지형에 따른 캐릭터의 발 위치와 관절 각도 보정을 실시간으로 계산하여 지형 형태에 발을 정확히 밀착시킴으로써 물리적 사실감을 높이려고 한다.

1.3. 서버

Registered I/O 모델을 적용한 고성능 서버 구조와 결합하여 대규모 데이터 처리, 낮은 지연시간을 보장하는 기술적 도전을 수행할 예정이다.

1.4. 결론

최종적으로 이러한 도전적인 기술적 기반 위에서 고품질의 그래픽과 실시간 상호작용 가능한 **3D** 멀티플레이 **PVP** 액션 게임을 만들어, 기술적 완성도와 시장성을 모두 갖춘 작품을 제작하는 것을 목표로 한다.

2. 게임 개요

2.1. 소개

2.1.1. 소개

게임 제목: EisenValor

장르: 3D 근접 액션-격투 하이브리드 + 멀티플레이어 협동 PVP

인원: 3 vs 3

시점: 3인칭 프리 백뷰, 3인칭 락온 솔더뷰

플레이 타임: 1판 / 20~35분

2.1.2. 재미 요소

1. 태세 전환 + 방향 읽기 기반의 심리전

중립태세와 전투태세를 전환하는 것으로 전장 이동과 교전을 플레이어가 선택하며 전투태세에서의 방향 읽기를 통해 상대의 선택을 고려해야 하는 심리전이다.

2. 타이밍 기반 전투로 심리전 강화

선 딜레이와 후 딜레이를 기반으로 시작되는 심리전

선 딜레이 중 피격 시 큰 데미지를 입는다. 따라서 플레이어는 딜레이를 생각하며 상대의 선택을 골라야 하며 여기서 페이크나 방어해제 등의 다양한 요소를 통해 다이나믹한 상황을 만들 수 있다.

3. 병사를 이끌거나 홀로 돌파하는 공성전의 전략성

각자의 스폰 지점에서 일정 시간마다 생성되는 병사와 공격팀이 운용하는 공성추를 중심으로 돌아가는 전장에서 상대의 장수를 공격하거나 공성전 중심 전장에서의 전략이 중요해진다.

2.1.3. 플랫폼 & 사양

해상도: 1920x1080, 2560x1440

평균 프레임: 60 FPS

플랫폼: Windows 10/11 64-bit

최소사양: RTX 3060, 메모리 16GB

권장사양: RTX 3080, 메모리 32GB

2.2. 규모

2.2.1. 주요 오브젝트

장수: 플레이어는 장수의 역할을 수행하며, 행동트리 기반으로 움직이는 NPC 장수도 플레이어와 동일한 역할 수행 가능

병사: FSM기반으로 움직이는 병사

2.2.2. 상호작용

공성추: 공격 진영에서 운용하는 공성추, 방어 진영의 성문을 부수는 역할

발리스타: 플레이어가 조종하여 투사체를 날릴 수 있음

2.3. 플레이

2.3.1. 카메라

1. 일반 모드 카메라

시점: 3인칭 프리 백 뷰

특징: 플레이어는 이동과 회전이 자유로우며, 주변 환경을 넓게 탐색할 수 있다. 이 상태에서는 달리기(Sprint)가 가능하여 전장 합류 및 거점 이동에 용이하다.

2. 전투 모드 카메라

시점: 3인칭 타겟 고정 솔더 뷰

특징: 전투 모드 전환 키(L-Ctrl) 입력 시, 시점이 타겟팅된 적으로 고정된다. 카메라는 1:1 교전에 집중할 수 있도록 줌인되거나 각도가 조정되며, 적을 화면 중앙에 두어 상/좌/우의 공격 및 방어

방향을 명확히 식별할 수 있게 돕는다.

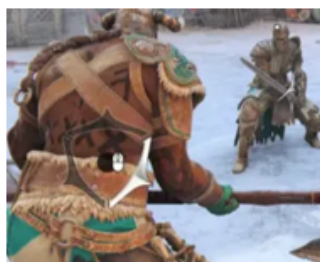
2.3.2. 컨트롤

키보드를 이용한 캐릭터의 이동/액션, 마우스를 이용한 3방향 스탠스 제어, 카메라 회전으로 구분된다.

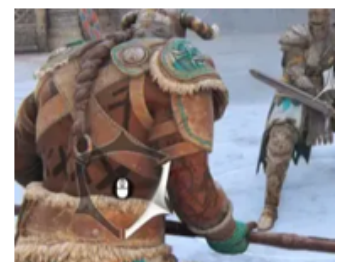
마우스 조작 및 스탠스 시스템

전투의 핵심 메커니즘으로, 마우스의 움직임에 따라 공격 및 방어 방향이 즉각적으로 결정된다.

3방향 스탠스: 상단(Top), 좌측(Left), 우측(Right)의 3가지 구역으로 나뉜다.



마우스가 움직인 방향으로 스탠스 변경



마우스가 안움직인다면 UI 중심으로 이동

조작 방식:

마우스를 움직인 방향으로 캐릭터의 공격 스탠스가 변경된다.

마우스 입력이 없을 경우, **UI** 중심으로 복귀하거나 마지막 입력 방향을 유지한다.

데드존 및 민감도:

UI 상단, 좌측, 우측의 각도와 민감도 설정에 따라 입력이 판정되며 특정 값 이하의 미세한 움직임은 무시하여 오입력을 방지한다.

공방 원리:

상대의 공격 방향과 동일한 스탠스를 유지하면 방어가 가능하다.

오직 전투 모드에서만 스탠스 조작이 가능하며, 일반 모드에서는 방어가 불가능하다.



액션 키 매핑 (Key Bindings)

구분	입력 키	동작	설명
공격	L-Click	약공격	일반적인 빠른 공격
	R-Click	강공격	약공격 대비 2배 강력한 공격력
	(L+R)-Click(전투)	범위 공격	넓은 범위를 타격하여 다수의 적 견제
	Wheel Click(전투)	무장해제	근접 상태에서 적의 방어를 강제로 해제
전술	E(전투)	페이크	공격 도중 행동을 취소하여 심리전 유도
	L-Ctrl	전투 모드 전환	중립/전투 모드 토글
이동	Space(중립)	구르기	길게 구르기
	Space(중립/전투)	회피 / 구르기(후방)	짧고 빠른 대쉬 / 후방 구르기
	L-Shift(중립)	달리기	빠르게 이동
	W / A / S / D	이동	wasd로 이동

3. 게임 특징

3.1. 태세 전환 기반의 3인칭 근접 전투

- 중립태세: 자유 카메라/이동 중심
- 전투태세: 락온 솔더뷰 + 교전 중심
- 전장 운영(이동/포지셔닝)과 교전(전투)을 명확히 분리하는 구조

3.2. 스탠스(좌/우/상) + 마우스 입력 기반의 심리전

- 마우스 이동으로 스탠스 변경, 스탠스 방향 방어
- 상대 읽기(방향/타이밍) 중심 전투를 형성

3.3. 선·후딜레이, 페이크, 카운터 규칙으로 만드는 타이밍

- 선딜/후딜 명세 + 페이크 조건 + 선딜 피격 페널티(카운터)
- 버튼 연타가 아니라 리스크/리턴 선택이 되게 함

3.4. 공성 + 점령전이 결합된 목표 기반 팀플레이(3v3)

- 공격/방어 비대칭 승리조건 + 거점 게이지 점령
- 전투뿐 아니라 목표 운영(호위/저지/점령)이 승패를 결정

3.5. 대규모 NPC 전장(장수 AI + 병사 FSM)로 전장 스케일 확보

- 장수: Behavior Tree
 - 공격, 방어, 심리전(무장해제, 페이크)
- 병사: FSM
 - 단순한 탐지, 추격, 전투 상태 전이

3.6. 전장 기믹 오브젝트(발리스타/회복소)로 전술 변수 제공

- 플레이어 조작형 발리스타, 고위력 투사체, 충돌 그룹 분리
- 교전 구도를 바꾸는 전술 카드로 기능

3.7. 현실적인 그래픽 연출

- 레이트레이싱과 볼륨 렌더링을 통한 현실성 있는 전장 경험

4. 게임 디자인

4.1. 맵 구조



<회복소>

- 점령한 후 체력을 서서히 회복할 수 있는 지역



<공성추>

- 파란색 길을 따라 이동



<성벽 수비 궁수>

- 계속해서 성벽에 리스폰



<성문>



<점령지>

4.2. 시스템

4.2.1. 전투태세 및 스탠스

1. 핵심 전투 메커니즘은 **3방향 스탠스(상단, 좌측, 우측)**시스템
2. 마우스 움직임을 통해 즉각적으로 공격 및 방어 방향을 전환, 상대의 공격방향과 동일한 스탠스를 유지하면 방어가 가능
3. 전투태세 (**L-Ctrl**)중에는 시점이 타겟팅된 적으로 고정되어 집중적인 **1:1** 교전

4.2.2. 딜레이 & 스테미나

1. 모든 행동에는 선 딜레이(**Wind-up**)와 후 딜레이(**Recovery**)가 존재
 - a. 선 딜레이 도중 피격 시 카운터(**Counter**) 판정으로 **1.5배**의 피해를 입는 리스크/리턴 구조
 - b. 선 딜레이 도중 페이크 키를 입력 시 현재 동작이 취소되며 바로 다음 동작 가능
 - c. 피격 시 피격 딜레이도 존재
2. 모든 행동은 스테미나를 소모하며, 소진 시 탈진 상태가 되어 딜레이가 **1.5배** 증가
 - a. 움직임 제외한 나머지 동작을 취하지 않으면 빠르게 회복

4.2.3. 봇 장수

- 플레이어가 부족할 경우 방에서 봇을 추가하여 **BT**로 돌아가는 장수 봇을 추가할 수 있다.
- 플레이어와 동일한 취급이며 리스폰 시간이 플레이어와 다르게 절반으로 적용된다.

4.2.4. NPC 병사

- 병사는 각 진영의 스폰 지점에서 생성된다.
- 양 진영의 병사는 기본적으로 공성추를 향해 움직이며 각 **FSM**에 맞는 상태전이로 전투를 택하거나 이동을 택한다.
- 플레이어는 주변 병사들에게 지시를 내릴 수 있다.
 - 모여라(**30초간 지속**): 주변 병사를 모을 수 있고 **30초간** 유지됨
 - 따라와라(**지속 유지**): 주변 일정 수 내의 병사가 플레이어를 따라다님
 - 저곳으로 가라: 플레이어가 보는 방향으로 이동

5. 개발 환경 및 활용 방법

5.1. 엔진 및 그래픽

- DirectX12(Ultimate) / HLSL2021
- DirectX Raytracing(DXR) 1.1
- C++23
- DLSS
- Custom Engine

5.2. 개발 도구

- Visual Studio 2022
- PIX for Windows
- RenderDoc
- Git

5.3. 라이브러리(NuGet)

- DXC
- FlatBuffers
- oneTBB(Intel TBB)
- RapidJSON

5.4. 리소스

- Blender
- FBX SDK
- Unreal Engine

5.5. 협업 및 관리

- Github
- Notion
- Google Docs
- Github Action

활용 방법

Notion과 Google Docs를 사용해 팀에서 사용해야 할 문서나 자료 조사, 기획 문서를 관리한다. Github와 Github Action을 사용하여 버전 관리와 CI환경을 구축해 개발 효율성을 올리고자 하였고 Git flow를 main - dev로 분리하고 squash merge PR 방식을 채택하여 커밋 내역을 깔끔하게 유지한다.

클라이언트 개발은 DirectX12 기반의 커스텀 엔진(프레임워크)를 사용한다. 디버깅을 편하게 하고자 assert와 로그를 활용한 일반적인 디버깅, 렌더링 파이프라인이나 리소스의 문제를 식별하기 위해 별도 래핑 처리를 적용했다. 추가적으로 PIX(정밀)나 RenderDoc(간단한 체크)를 활용한 그래픽 디버깅 및 프로파일링을 하여 성능 최적화를 진행할 예정이다.

기술적 제약 사항으로 DXR 1.1 사용 시 intellisense 미지원 이슈가 있어 공식 문서를 교차 검증하며 적용해야 하고, 셰이더 언어는 HLSL 2021 버전을 사용한다.

리소스는 에셋 구매 및 Blender를 활용한 자체 수정을 병행하며 FBX 포맷을 표준으로 사용한다. 맵 추출이나 애니메이션 편집 등은 엔진에서 유용한 부분을 활용할 수 있도록 자체 익스포터를 만들어 제작하여 본 게임에 적용할 것이다.

6. 타 게임과의 비교(경쟁력)

포 아너

- 포 아너는 3인칭 영웅 기반 근접 격투로, 3방향 가드/스탠스 중심의 **Art of Battle** 전투가 핵심
- **EisenValor**는 유사하게 3방향 가드/스탠스 심리전을 채택하되, 공성/점령 목표전과 전장(다수 NPC) 운영을 결합해 결투 + 목표 수행 중심으로 게임의 메인 흐름이 존재
- **DX12** 기반 렌더링 파이프라인을 전제로 시각적 품질(하이브리드 RT, Volumetric 등)을 연구 목표로 삼아, 기술적 차별화

워 헤이븐

- 대규모 팀 **PVP**로 중세 판타지 근접전이 핵심, 캐주얼한 전장 변수 특징
- 이미 서비스 종료로 기존 경쟁작으로 레퍼런스의 성격으로 비교
- **EisenValor**에선 **3 vs 3**으로 줄이고 고속전의 전투를 강조하며 공성 목표로 전략성을 두어 차별화

컨커러스 블레이드

- 영웅 조작 + 부대 지휘의 온라인 전술 액션, 대규모 공성전에서 수많은 병력/공성장비 운용 핵심
- **EisenValor**는 부대 지휘보단 개인의 전투 능력이 중심, 병사에게 명령만 간단히 내리고 대부분 플레이어 스스로 해야하는 것이 차별화

진 삼국무쌍

- 핵 앤 슬래시이며 **1 vs** 다수를 압도하는 전장의 쾌감이 중심
- **EisenValor**는 대규모 전장의 느낌을 NPC 병사로 일부 채택하되, 핵심을 실시간 **PVP** 근접 전투로 전환해 경쟁력을 둠
- 싱글 중심 무쌍이 아닌 대전 + 팀 전략 + 목표 수행의 성격으로 구조적 차별화

7. 개인별 준비 현황

7.1. 이윤상(클라이언트)

- C/C++ 프로그래밍, STL
- 윈도우 프로그래밍, 컴퓨터 그래픽스, 게임수학
- 게임엔진1(unity)
- 컴퓨터 구조
- 운영체제
- 데이터베이스
- OpenGL 게임 프레임워크 제작 경험
- 소프트웨어 레이 트레이싱 제작 경험
- DXR 1.1 사용 가능
- 인공지능 부전공

7.2. 윤성열(서버)

- C/C++ 프로그래밍, STL
- 윈도우 프로그래밍, 게임수학
- 컴퓨터 구조
- 컴퓨터 그래픽스
- 운영체제
- 게임서버 프로그래밍
- 고급 멀티쓰레드 프로그래밍, 멀티코어 프로그래밍
- 네트워크 게임 프로그래밍
- 인공지능
- 데이터베이스

7.3. 이다건(클라이언트)

- C/C++ 프로그래밍, STL
- 윈도우 프로그래밍
- 선형대수학
- 컴퓨터 그래픽스
- 게임수학
- 2D 게임 프로그래밍
- 컴퓨터 구조
- 게임엔진1(Unreal)
- 3D게임프로그래밍1, 3D게임프로그래밍2(DirectX12)

8. 중점연구분야

8.1. 이윤상

1. 현실적인 렌더링을 위한 **Clustered Forward+ & DXR** 기반 **ReSTIR DI/GI** 레이트레이싱

Raster DI, ReSTIR DI, ReSTIR GI를 프레임 상황에 맞춰 적절히 운영하는 적응형 Hybrid구조로 View Frustum 3D grid로 분할하는 것을 통해 클러스터별 리스트를 구축한다.

Raster DI는 투명/특수 Material들의 처리 및 안정적 DI(직접광, 키 라이트)를 처리하고 DXR파이프라인으로 ReSTIR DI / GI를 수행 이후 디노이징하여 DI와 GI의 항을 중복 없이 합산하여 결과를 통합한다. 이를 통해 현실적인 렌더링을 구현하고자 한다.

Raster는 사람피부(SSS근사), 헤어, 금속반사, VFX, 데칼 등 전투 연출을 담당하고 RayTracing은 햇불/화염 기반 동적 조명에서 정확한 빛과 환경의 연출을 담당하여 현실감을 더할 것이다.

2. Volumetric Rendering

Forward+를 연계하여 3D grid를 사용해 log z 슬라이스 경계 기준으로 step을 이어가며 레이마치를 하는 Volume Render를 추가하고, 이때 이 step은 상한을 둘 예정이다. (Volumetric 적분 Artifact 방지)

현실적인 렌더링을 강화하기 위해 Volumetric Rendering을 적용한다: 햇불/화염의 Volumetric 산란(광량 분포), 강·웅덩이의 수중 색 흡수(깊이감), 안개/연무의 산란 기반 시야 감쇠를 구현하고, 볼륨 조명에 동적 광원(햇불 등)이 실시간으로 반영되도록 한다.

3. Performance Tuning

NRD 혹은 DLSS 3.5 Ray Reconstruction을 통해 레이트레이싱 결과를 디노이징한다. TAAU 혹은 DLSS Super Resolution를 통한 업스케일링을 한다.

8.2. 윤성열

1. Registered I/O 모델을 이용한 멀티 스레드 서버

멀티플레이 게임 서버는 다수의 클라이언트 요청을 실시간으로 처리해야 하므로, 네트워크 I/O 성능과 확장성이 핵심적인 요소로 작용한다.

Registered I/O(RIO) 모델은 사용자 영역에서 네트워크 버퍼를 사전 등록하여 I/O 처리 오버헤드를 최소화할 수 있는 구조를 제공하지만, 실제 게임 서버 개발 사례는 드물다.

이에 본 졸업작품에서는 Registered I/O 모델을 기반으로 한 멀티 스레드 서버를 설계·구현함으로써 게임 서버 분야에서의 적용 가능성을 검증하고자 한다

8.3. 이다건

1. 중요도 기반 애니메이션(Animation LOD)

카메라 초점, 거리 기반, 화면 점유율(시/공간적) 등 다양한 요소에 따라 중요도를 정하고 그에 따른 애니메이션 LOD를 구현한다.

중요도가 낮은 애니메이션은 프레임을 건너뛰고 자연스럽게 보간하도록 설계한다.

2. IK를 활용한 지형에 맞는 발 위치 만들기

RayCasting으로 수집한 지면의 높낮이와 경사 데이터를 활용해 발, 무릎, 골반의 관절 각도를 실시간으로 역계산한다.

불규칙한 지형에서도 캐릭터의 발이 지면에 정확히 밀착되게 하여 물리적 사실감과 몰입감을 높인다.

3. 추가 구현 목표 : Fluid Simulation Ripple (GPU)

물웅덩이를 밟았을 때 파동이 퍼져나가는 것을 Compute Shader로 구현한다.

Compute Shader를 활용해 2차원 파동 방정식을 병렬 연산하고 그 결과를 실시간 노멀 맵으로 변환하여 물 표면에 적용한다.

9. 기술적 강점

9.1. 클라이언트

Clustered Forward+는 라이트 그룹을 생성하는 구조적 특성 때문에 **DXR**을 활용한 **ReSTIR DI**는 기술적 접점이 많아 시너지를 낼 수 있다. 또한 **Volumetric Render** 구현 시에도 공간 분할을 별도로 할 필요 없이 재사용할 수 있다는 이점이 있다. 무엇보다 **Forward** 렌더링 기반이므로 **Deferred** 방식과 다르게 재질 표현에 제약이 적고 헤어, 금속, **Subsurface scattering**, 데칼 등의 묘사로 품질을 높일 수 있다.

따라서 핵심 구조 하나만 구현해도 여러 기술로 확장하기 용이하고 그 기술 하나하나의 렌더링 품질 상승 효과가 매우 높기 때문에 이 방식을 채택한다. 특히 우리 게임은 실사 그래픽을 중심으로 하면서도 개발 기한에 제한이 있기에, 단일 구조를 통해 여러 마일스톤을 효율적으로 달성할 수 있는 구조가 필수적이다. 이를 위해 **RenderPass** 형태로 기술들을 구분하여 추상화하고 개발을 이어갈 것이다.

실사 그래픽인 만큼 애니메이션 또한 중요한 요소이다. 수많은 **NPC**와 캐릭터들의 애니메이션이 존재하는 프로젝트이므로 중요도 기반의 애니메이션 **LOD**를 통해 게임 성능을 최적화하고, **IK**를 활용한 지형 구조에 맞는 발 붙이기 등을 통해 캐릭터 애니메이션의 품질을 높일 것이다.

9.2. 서버

RIO 모델이 기존 네트워크 모델과 차별화 되는 부분은 **I/O**에 사용하는 메모리 관리 방식이다. 기존 **WSASend**, **WSARecv**를 비동기로 사용할 경우 요청할 때 마다 메모리를 준비하는 방식이라면, **RIO**는 미리 준비된 메모리만 사용하는 방식이다.

기존 **WSASend**나 **WSARecv**를 호출할 땐 데이터가 담긴 버퍼의 포인터를 넘긴다. 이 포인터는 해당 프로세스가 사용하고 있는 가상 메모리의 주소이며, 운영체제는 이 비동기 요청을 받을 때마다 커널은 다음과 같은 무거운 작업을 매번 반복한다.

1. 해당 버퍼의 가상 메모리 주소가 유효한지 검사한다
2. 해당 가상 메모리가 물리 메모리에 존재하는 확인한다
3. **Page -Lock**
4. **I/O** 완료 후 **Page Unlock**

RIO는 **I/O**에 사용할 버퍼를 미리 등록해 두고, 그 물리 메모리를 계속 **pinned** 상태로 두기 때문에 매 **I/O** 마다 **page-lock/unlock**을 반복할 필요가 없다. 따라서 **CPU**가 사용하는 시간을 줄여서 처리량을 높였고, 또한 **IOCP**처럼 **OS**가 스레드를 깨우는 반응이 아닌 **Polling** 방식으로 **CompletionQueue**를 계속 감시하게 코드를 구현하여 스레드 컨텍스트 스위칭을 제거함으로써 시간을 단축시킬 수 있다.

10. 개발 일정 및 구성원 역할 분담

이윤상(클라이언트)	
윤성열(서버)	
이다건(클라이언트)	

	1월	2월	3월	4월	5월	6월
클라이언트 코어 구축						
렌더 베이스 구축						
Clustered Forward+						
ReSTIR DI GI						
Volumetric Render						
Performance Tuning						
서버 입출력 제작						
게임 콘텐츠 구현						
DB 연동						
로비 구현						
애니메이션 기반 작업						
애니메이션 LOD						
IK 지형 적용 시스템						
GPU 유체 시뮬레이션						

11. 참고자료

RIO 관련 참조 논문

<https://www.slideshare.net/slideshow/windows-registered-io-rio/24250865>

<https://scienceon.kisti.re.kr/commons/util/originalView.do?cn=JAKO202007650437083&oCn=JAKO202007650437083&dbt=JAKO&journal=294576>

<https://koreascience.kr/article/JAKO202007650437076.pdf>

Clustered Forward+ + DXR 논문:

<https://www.cg.tuwien.ac.at/research/publications/2025/elagabani-2025-ieh/elagabani-2025-ieh-bachelor-thesis.pdf>

Unity - Scripting API:

<https://docs.unity3d.com/6000.1/Documentation/ScriptReference/index.html>

EnTT Github:

[skypjack/entt](https://github.com/skypjack/entt): Gaming meets modern C++ - a fast and reliable entity component system (ECS) and much more

Raytracing:

<https://github.com/NVIDIA-RTX/RTXDI>

ReSTIR GI:

https://research.nvidia.com/publication/2021-06_restir-gi-path-resampling-real-time-path-tracing

DXR 1.1 문서:

<https://microsoft.github.io/DirectX-Specs/>